

PhishGuard: An Automated Phishing Email Analysis and Risk Scoring System

Dana Salem
Department of Computer Science
Princess Sumaya University for Technology
Amman, Jordan
dan20220177@std.psut.edu.jo

Laila Sadideen
Department of Computer Science
Princess Sumaya University for Technology
Amman, Jordan
lai20220432@std.psut.edu.jo

Jameel Al Shurafa
Department of Computer Science
Princess Sumaya University for Technology
Amman, Jordan
jam20220661@std.psut.edu.jo

Ammar Odeh
Department of Computer Science
Princess Sumaya University for Technology
Amman, Jordan
a.odeh@psut.edu.jo

Abstract— Phishing remains a serious cyber threat because it combines social engineering with technical indicators such as spoofed sender identities, suspicious domains, deceptive URLs, and risky attachments. This paper presents PhishGuard, an automated and explainable phishing email analysis system. The proposed model parses .eml files, analyzes headers, domains, URLs, and attachments using static security tools, then applies a weighted risk-scoring engine to classify emails as Allow, Quarantine, or Block. The system was evaluated on a hybrid dataset of 240 samples and achieved 80.00% overall accuracy, 87.88% precision, and 96.67% recall, while allowing only two malicious emails to pass. The results show that transparent multi-stage static analysis can support practical phishing triage without relying on opaque black-box models.

Keywords—Phishing detection, email security, risk scoring, static analysis, explainable cybersecurity, threat triage

I. INTRODUCTION

Phishing attacks remain a common and damaging cybersecurity problem because they exploit both technical weaknesses and human trust. Attackers may use fake sender identities, convincing messages, malicious links, or attachments to trick users into sharing sensitive information or executing harmful content. Recent phishing studies show that attacks continue to evolve across several forms, including email phishing, spear phishing, clone phishing, smishing, vishing, and pharming [1]-[3]. This variety makes phishing difficult to stop using a single indicator or a single defensive layer.

Many existing detection approaches rely on machine learning, deep learning, or isolated URL and content features. These methods can be effective, but they are often difficult to explain to security analysts, especially when a decision must be justified. PhishGuard was designed to address this gap by using a transparent workflow that analyzes several parts of the email and converts the findings into a clear risk score. The goal is not only to classify an email, but also to show why the classification was made.

The main contributions of this work are as follows: 1) an automated workflow for analyzing .eml files, 2) modular analysis of headers, domains, URLs, and attachments, 3) integration of common security tools inside a Python-based pipeline, 4) an explainable risk-scoring model with contextual

penalties, and 5) evaluation on a hybrid dataset containing safe and malicious email scenarios.

II. RELATED WORK

Previous research has examined phishing from several directions. Survey studies summarize phishing types, attack vectors, and technical countermeasures, showing that phishing is not limited to one medium or one behavior [1]-[3]. Other work focuses specifically on email filtering and phishing detection techniques, including blacklist-based methods, heuristic rules, feature extraction, and machine learning models [4], [5]. These studies support the need for multi-stage detection instead of depending on a single sign such as sender address or URL appearance.

URL-based detection is one of the most common research directions. Sahingoz et al. used natural language processing and machine learning features to detect phishing URLs in real time [6]. Deep learning approaches have also been used for phishing email detection, where models analyze email structure and text to classify suspicious messages [7]. Another related direction is multi-filter phishing detection, where several checks are combined to improve coverage across different attack behaviors [8]. Compared with these methods, PhishGuard emphasizes explainability and operational clarity. Rather than training a black-box model, it combines interpretable indicators and reports the score contribution of each finding.

III. PROPOSED MODEL

A. System Architecture Description

The proposed model begins when the user uploads an email file in .eml format. The Email Parser Module extracts the important email components, including headers, body, URLs, sender domain, and attachments. After parsing, the email is processed by four analysis modules. The Header Analysis Module checks sender authenticity, Reply-To mismatches, SPF, DKIM, DMARC, suspicious mailers, urgency keywords, and routing hops. The Domain Analysis Module checks DNS resolution, domain age, registration information, and typosquatting indicators. The URL Analysis Module inspects URL patterns, redirects, IP-based links, shortened URLs, suspicious TLDs, deceptive subdomains, and phishing keywords. The Attachment Analysis Module

inspects PDF and Office files for risky structures such as macros, embedded actions, and suspicious interactive content.

After the modules finish, their outputs are collected into a Findings and Indicators Collection component. The Risk Scoring Engine then assigns weighted values to each

suspicious indicator and adds extra penalties for dangerous combinations. Finally, the Decision Engine maps the total score into Allow, Quarantine, or Block. Fig. 1 shows the full architecture of the proposed system and illustrates how the input, analysis, scoring, and reporting layers are connected.

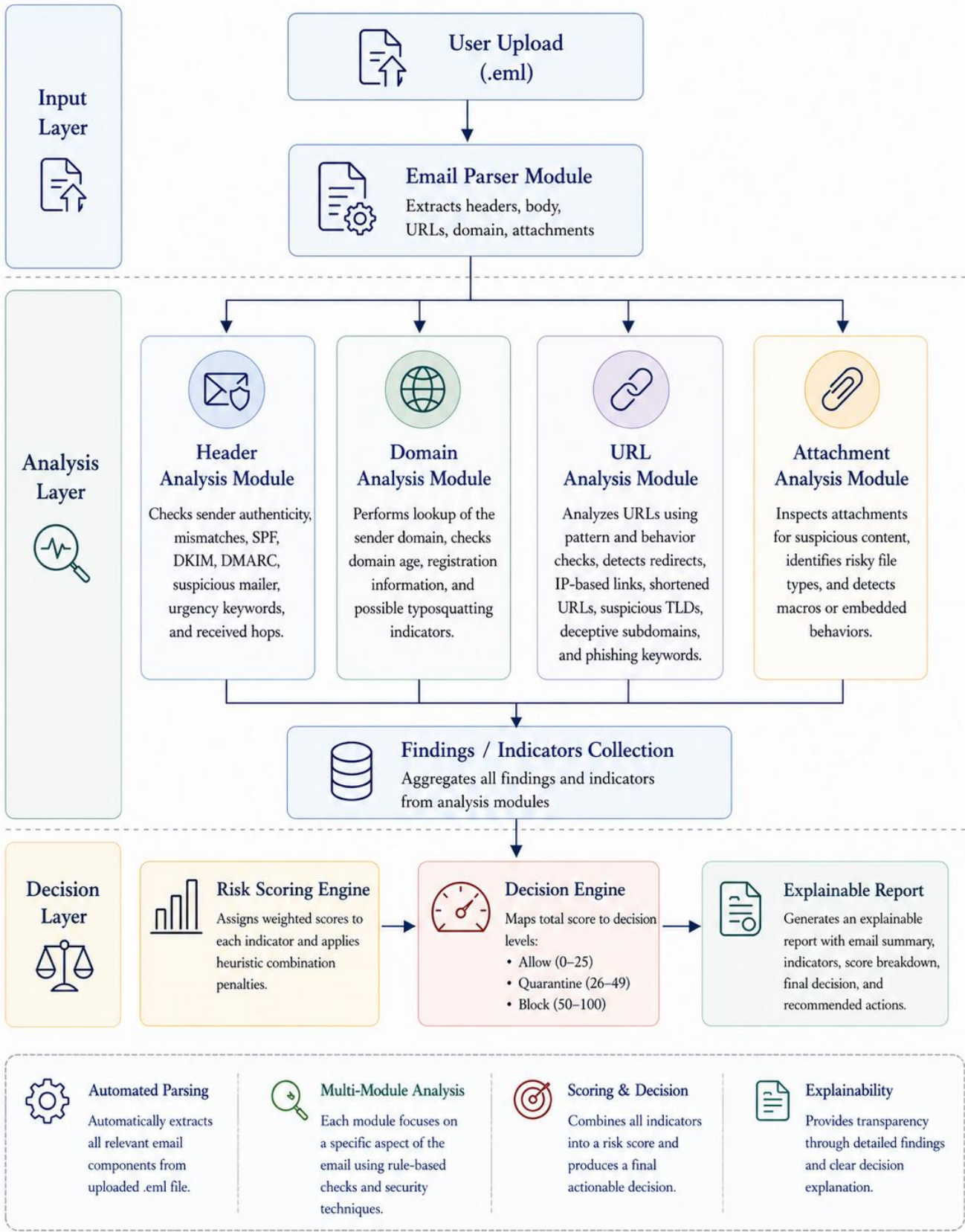


Fig. 1. System architecture of the proposed phishing email analysis and risk scoring model.

B. Tools and Technologies

Python controls the workflow and connects the analysis modules. The built-in email and email.policy libraries parse the uploaded .eml file and extract headers, body text, URLs, and attachments. For domain analysis, nslookup checks whether the sender domain resolves correctly, while whois retrieves registration information such as domain age and registrar details. For URL analysis, curl is used to observe link response behavior and redirects. For attachment analysis, PDFiD checks PDF files for suspicious objects, while oleid and olevba inspect Office files for macros or suspicious VBA content.

C. Risk Scoring Model

The scoring model converts suspicious findings into one numerical score. First, all findings from the header, domain, URL, and attachment modules are collected into one set:

$$F = H \cup D \cup U \cup A \quad (1)$$

where F represents all findings, H represents header findings, D represents domain findings, U represents URL findings, and A represents attachment findings. Each finding receives a weight according to its severity:

$$\text{Raw Score} = \sum w(f_i) \quad (2)$$

The system also applies contextual combination penalties when two or more indicators appear together. This is important because some phishing behaviors become more dangerous when combined, such as sender spoofing with a short URL or urgency language with a suspicious attachment:

$$\text{Combo Score} = \sum c_j \quad (3)$$

The final score is capped at 100 to keep the output within a clear range:

$$\text{Final Score} = \min(\text{Raw Score} + \text{Combo Score}, 100) \quad (4)$$

The weighted scoring idea is aligned with the practical scoring approach used in Apache SpamAssassin, which combines heuristic and statistical tests for email analysis [9]. The use of contextual combinations is also consistent with the MITRE ATT&CK view of adversary behavior as a set of related tactics and techniques rather than isolated actions [10]. The final score is mapped to Allow for 0-25, Quarantine for 26-49, and Block for 50-100. These thresholds were selected to create a conservative triage model and were guided by risk severity concepts from CVSS and NIST risk assessment guidance [11], [12]. Email trust concepts such as sender authentication and domain spoofing were also guided by NIST trustworthy email recommendations [13].

IV. SIMULATION SETUP

The practical testing was performed in Kali Linux because it provides useful command-line tools for domain, URL, and attachment analysis. Python scripts were used to organize the dataset, run the tools, collect indicators, calculate the risk score, and record the final decision. The dataset contained 240 email samples arranged into an eight-case test matrix, with 30 samples in each case. Email content was based on the Apache SpamAssassin public corpus, and attachment samples were taken from the CIC-Trip4Phish 2025 dataset [14], [15].

The eight test scenarios were: clean baseline, header suspicious, URL only safe, URL only malicious, attachment only malicious, full safe, full attack, and mixed stealth. This structure helped isolate specific variables inside the scoring engine. For example, the header suspicious case tested sender identity problems without URLs or attachments, while the full attack case tested multiple vectors at the same time. This design made it possible to evaluate both simple and complex phishing behaviors.

V. RESULTS AND ANALYSIS

A. Case Study Results

Three detailed cases were used to demonstrate the decision process. In the Allow case, the email contained a trusted domain, a clean GitHub URL, and an Office attachment without active macros. The final score was 0, so the decision was Allow. This case shows that the system can pass normal business-like communication without unnecessary blocking.

In the Quarantine case, the email had no links and no attachments, but the Header Analysis Module detected an SPF failure and a Reply-To mismatch. These findings produced a score of 41, which placed the email between the Allow and Block thresholds. The decision was therefore Quarantine. This case is important because it shows that text-only social engineering can still be treated as suspicious even when no malware file exists.

In the Block case, the email combined a fabricated sender domain, a deceptive Microsoft-like URL, and a suspicious PDF containing an interactive form. These findings produced a capped score of 100, so the decision was Block. This case shows the value of combining several indicators, because the final decision is stronger when domain, URL, and attachment evidence all point to malicious behavior.

B. Overall Dataset Results

When tested on all 240 samples, PhishGuard achieved 80.00% overall accuracy, 87.88% precision, and 96.67% recall. The system was especially effective in the extreme scenarios, where full safe and full attack samples were classified correctly. Only two malicious emails were incorrectly allowed, which reflects a strict security posture. Most incorrect decisions appeared in boundary cases, especially where static attachment analysis could not fully detect heavily hidden or obfuscated behavior.

The results suggest that the proposed architecture is useful for transparent phishing triage. It does not simply return a label; it reports which modules produced suspicious indicators and how these indicators affected the total score. This makes the output easier to explain to administrators and easier to audit than black-box models.

VI. DISCUSSION

The main strength of PhishGuard is that it combines several lightweight checks into one explainable decision. Header analysis alone may miss suspicious links, URL analysis alone may miss malicious attachments, and attachment analysis alone may not identify sender spoofing. By combining these modules, the system builds a clearer picture of the email. The risk-scoring model also helps handle

mixed cases, where an email is not completely safe but also not obviously malicious.

The main limitation is that the current model relies on static analysis. It checks the structure and indicators of files, but it does not execute attachments in a sandbox. As a result, deeply obfuscated, password-protected, or nested malicious content may evade detection. Another limitation is that the body text is not analyzed using natural language processing, so advanced text-only social engineering may still require human review. Future improvements should include dynamic sandboxing, recursive archive extraction, reputation APIs, YARA rules, and NLP-based body analysis.

VII. CONCLUSION

This paper presented PhishGuard, an automated phishing email analysis and risk-scoring system. The system parses .eml files, analyzes headers, domains, URLs, and attachments, then produces an explainable score and final decision. The model was evaluated on a 240-sample hybrid dataset and achieved 80.00% overall accuracy, 87.88% precision, and 96.67% recall. The results also showed strong behavior in clear safe and full attack scenarios. The work demonstrates that transparent static analysis can provide practical phishing defense when explainability and evidence-based decisions are important.

REFERENCES

- [1] Z. Alkhalil, C. Hewage, L. Nawaf, and I. Khan, "Phishing attacks: A recent comprehensive study and a new anatomy," *Frontiers in Computer Science*, vol. 3, 2021.
- [2] K. L. Chiew, K. S. C. Yong, and C. L. Tan, "A survey of phishing attacks: Their types, vectors and technical approaches," *Expert Systems with Applications*, vol. 106, pp. 1-20, 2018.
- [3] R. Alabdan, "Phishing attacks survey: Types, vectors, and technical approaches," *Future Internet*, vol. 12, no. 10, 2020.
- [4] A. Almomani, B. B. Gupta, S. Atawneh, A. Meulenberg, and E. Almomani, "A survey of phishing email filtering techniques," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2070-2090, 2013.
- [5] M. Khonji, Y. Iraqi, and A. Jones, "Phishing detection: A literature survey," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2091-2121, 2013.
- [6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," *Expert Systems with Applications*, vol. 117, pp. 345-357, 2019.
- [7] Y. Fang, C. Zhang, C. Huang, L. Liu, and Y. Yang, "Phishing email detection using improved RCNN model with multilevel vectors and attention mechanism," *IEEE Access*, vol. 7, pp. 56329-56340, 2019.
- [8] G. Sonowal, "PhiDMA: A phishing detection model with multi-filter approach," *Journal of King Saud University - Computer and Information Sciences*, vol. 32, no. 1, pp. 99-112, 2020.
- [9] The Apache Software Foundation, "Apache SpamAssassin," 2024. [Online]. Available: <https://spamassassin.apache.org/>
- [10] MITRE Corporation, "MITRE ATT&CK," 2024. [Online]. Available: <https://attack.mitre.org/>
- [11] FIRST, "Common Vulnerability Scoring System v3.1: Specification Document," Forum of Incident Response and Security Teams, 2019. [Online]. Available: <https://www.first.org/cvss/v3.1/specification-document>
- [12] Joint Task Force Transformation Initiative, "Guide for Conducting Risk Assessments," NIST Special Publication 800-30 Revision 1, Sep. 2012.
- [13] S. Rose, S. Nakassis, and M. Santos, "Trustworthy Email," NIST Special Publication 800-177 Revision 1, Feb. 2019.
- [14] Apache Software Foundation, "Apache SpamAssassin Public Corpus." [Online]. Available: <https://spamassassin.apache.org/old/publiccorpus/>
- [15] Canadian Institute for Cybersecurity, "CIC-Trip4Phish 2025 Dataset," University of New Brunswick, 2025. [Online]. Available: <https://www.unb.ca/cic/datasets/trip4phish2025.html>